

Highlights

Text-to-SQL in Process Mining: Revisiting Strategies and Expanding the Discussion

- Evaluate three LLMs on text-to-SQL with zero/few-shot prompts and two representations
- Assess usability and feasibility of text-to-SQL in process mining
- Analyze SQL pattern templates and SQL complexities
- Analyze SQL complexities and information retrieval in process mining domain

Text-to-SQL in Process Mining: Revisiting Strategies and Expanding the Discussion

Abstract

Accessing databases through natural language has the potential to broaden information retrieval, making it available to users without SQL expertise. The generation of SQL statements from natural language utterances, referred to as the text-to-SQL task, can also benefit the field of process mining, which leverages event logs from business process executions to provide valuable insights. Yet, despite this potential, the text-to-SQL task in process mining domain remains largely unexplored. In this paper, we extend prior work by evaluating text-to-SQL in process mining using *text₂SQL₄PM*, a bilingual domain-specific dataset created to serve as a benchmark for text-to-SQL implementations. We assessed three large language models under different prompt strategies and representations, and performed a detailed analysis of the results to provide a clearer view of the usability and feasibility of applying text-to-SQL in this domain. Our findings underscore the complexity that text-to-SQL entails when guided by prompt engineering, and discuss this complexity in an evolutionary and domain-informed manner, offering contributions both for researchers aiming to advance technical solutions for the task and for practitioners interested in building practical applications upon them.

Keywords: Text-to-SQL, Process mining, Large language models, Prompt engineering, Benchmark

1. Introduction

The text-to-SQL task consists of generating an executable SQL statement from a natural language utterance, enabling the retrieval of information from a database according to the user’s intent [1]. Querying databases in natural language has the potential to make information retrieval accessible to users without knowledge of SQL syntax or commands. Advances in deep learning

have made it possible to address this task using encoder–decoder architectures [2, 3] and tunable pre-trained language models [4, 5, 6]. More recently, large language models (LLMs) have emerged as novel implementations for the text-to-SQL task, leveraging various prompt engineering strategies [7, 8].

Regardless of the implementation, three benchmarks [9, 10, 3] are commonly used for tuning and evaluating such models. Although these benchmarks span multiple domains—including cars, finance, pets, sports, and flights, it remains unclear how model implementations transfer to specialized domains with specific information retrieval requirements. Process mining represents one such case. As an interdisciplinary field, process mining analyzes data recorded in event logs, which capture the execution of business processes under study [11]. From event logs, users derive actionable insights by applying either specialized algorithms or information retrieval techniques. Although event logs are not inherently stored in relational databases, they can be transformed into relational formats, enabling SQL-based retrieval and supporting the extraction of valuable process insights.

In the process mining literature, LLMs have been explored for the generation of SQL statements using prompts enriched with business process information and process mining-specific instructions [12]. Although this approach aimed to evaluate the feasibility of LLMs for information retrieval, the reliance on advanced prompts makes it difficult to clearly assess their inherent capabilities and limitations in this domain. In previous work [13], basic prompt strategies were briefly evaluated across three large language models—GPT-3.5 Turbo [14], Gemini-1.0 Pro [15], and Llama3-8B Instruct [16], using a process mining dataset called *text₂SQL₄PM* and considering only the English language. On that occasion, we investigated the usability and feasibility of text-to-SQL implementations in process mining domain by applying zero-shot and one-shot prompt strategies to the domain-specific *text₂SQL₄PM* dataset, following the DAIL-SQL methodology [7]. We also introduced a set of SQL pattern templates inspired by NL2SQL360 [17] to support the analysis of the results, with a particular focus on SQL complexity.

In this paper, we revisit the language models and prompt engineering strategies to expand both the results and the scope of the analysis, as follows:

- Extension of the evaluation with the same LLMs used previously, now including two additional prompt strategies (three-shot and five-shot, beyond the zero-shot and one-shot considered before).

- Incorporation of the Portuguese language into the experiments, allowing fuller exploitation of the bilingual nature of the *text₂SQL₄PM* dataset.
- Revision of the SQL complexity analysis guided by templates, with a modified visualization of results that provides a slightly different perspective compared to the previous study.
- Expansion of the discussions, offering deeper insights supported by analyses more closely aligned with the process mining domain.

The remainder of this paper is structured as follows. Section 2 provides an overview of the basic concepts of text-to-SQL benchmarks and implementations, following the definitions of process mining and event logs. Section 3 discusses related work. Section 4 details the methodology, including the dataset, implementation details, and evaluation strategies. Section 5 presents and analyzes the results. Finally, Section 6 summarizes the findings and outlines the directions for future research.

2. Preliminaries

In this section, we present the fundamental concepts required to enable understanding of the implementation strategies for the text-to-SQL task, as well as the corresponding discussions, particularly as applied to problems in the field of process mining, developed in this paper.

2.1. Text-to-SQL task, benchmarks and implementations

Natural language processing encompasses a range of tasks, linguistic phenomena, and computational methods that allow natural language, with some degree of formalization, to be understood and processed by computers [18]. Beyond traditional tasks such as machine translation [19], research has advanced in mapping natural language utterances into formal, structured representations [20]. Within this scope emerges the investigation of the generation of SQL statements from natural language utterances, referred to as the text-to-SQL task.

Formal definition. In the text-to-SQL task, a natural language utterance is taken as input and a SQL statement is generated as output, which can then be executed on a database. According to [21], this task can be formally defined as the mapping $c : U \rightarrow S$, in which U is a universe of natural language utterances, S a universe of SQL statements, and $c(u) \rightarrow s$ a conversion procedure from elements $u \in U$ to elements $s \in S$. This definition is complemented by assertions arising from the expressiveness of the languages involved [21], which need to be considered in strategies designed to solve text-to-SQL problems:

- $U' \subset U \mid \forall (u'_i, u'_j) \in U'$ and $i \neq j$, u'_i and u'_j are paraphrases in natural language, meaning they express the same intent;
- $S' \subset S \mid \forall (s'_i, s'_j) \in S'$ and $i \neq j$, s'_i and s'_j are equivalent SQL statements, meaning that they produce the same result when executed;
- if $c(u'_i) \rightarrow s'_i$, then $c(u'_i) \rightarrow s'_j$, $c(u'_j) \rightarrow s'_i$ and $c(u'_j) \rightarrow s'_j$ for any i, j .

Solution design. Solutions for implementing text-to-SQL should focus on two main central challenges: *representing the database schema*, which together with the utterance constitutes the input to the task; and *performing schema linking*, that is, aligning terms in the utterance with the relations, attributes, and values of the schema.

Benchmarks. At present, there are three well-known text-to-SQL benchmarks for evaluating text-to-SQL implementations: WikiSQL [3], Spider [10] and Bird [9]. WikiSQL is the simplest benchmark according to schema structure, with only a single table and simple SQL statements. The Spider benchmark is more robust than WikiSQL, consisting of SQL statements with complex clauses and a schema structure with tables containing relationships between them. The Bird benchmark contains a more robust dataset than Spider, with pairs of utterances that need domain knowledge to be answered, and SQL statements with complex clauses and function operators.

Implementations. For a long time, most text-to-SQL implementations that attained competitive results on the WikiSQL and Spider benchmarks were based on supervised fine-tuned encoder-decoder neural network architectures. Initially, the utterance input was encoded with a bidirectional LSTM in Seq2SQL [20] and SQLNet [2], and the decoder, conditioned on the representation produced by the encoder, generates SQL statements in Seq2SQL

[20] or fills the slots for each SQL clause in SQLNet [2]. For Spider benchmark, which also requires schema representations of the database as input, a graph neural network was used to encode the utterance and the schema representation in GNN [22] and RATSQ [6], and the decoder, conditioned on that representation, generates SQL statements. Other approaches used pre-trained language models such as GAP [5] and T5 [23] to produce contextualized input representations and, using an encoder-decoder architecture like RAT-SQL [6] or RESDSQL [4], generates SQL statements.

Although early deep learning approaches to text-to-SQL treated *schema representation* and *schema linking* as distinct challenges, with the introduction of cross-domain benchmarks, however, these tasks began to be addressed jointly [6, 4]. More recently, large language models have further blurred this distinction by processing both the utterance u and the database schema within a prompt, selectively emphasizing the schema elements most relevant to the query. Among these implementations, strategies such as DAIL-SQL [7] and DIN-SQL [8] exemplify how prompt design can guide the generation of SQL statements.

2.2. Process mining and event logs

Process mining is positioned at the intersection of two fields of knowledge: data science and process science [11]. The primary source of information for process mining is event logs, which serve as the foundation for applying analysis strategies to identify workflow bottlenecks, anticipate performance issues, assess conformance and compliance, recommend countermeasures, and optimize business processes [24].

Event log. An event log records a sequence of *events* related to the execution of activities in a business process under analysis. This business process can be executed multiple times, with each execution referred to as a *process instance*, which is commonly referred to as *case*. Each case is unique and can be identified by a *caseID*. Traditionally, each event is related to only one case, and a case refers to a unique sequence of events. In addition to information about the case to which each event is linked, other attributes can be recorded for each event, such as the activity performed (an essential attribute), the resource (person or system) involved in its execution, and the associated cost. A mandatory requirement is that the order of events within a case reflects the order of their occurrences over time, which is often recorded in the event log by the *timestamp* attribute [11]. Figure 1 shows

a summarized example of an event log, referring to the business process of the reimbursement procedure at a Dutch university¹, which is used in the *text₂SQL₄PM* dataset [21].

event_id	case_id	activity	timestamp	resource	cost
1	declaration 76457	Start trip	04-10-2016 22:00:00	Linda	2
2	declaration 76457	End trip	04-10-2016 22:00:00	Linda	3
3	declaration 76457	Permit submitted by employee	06-04-2017 11:32:00	Linda	5
4	declaration 76457	Permit final approved by employee	06-04-2017 11:32:28	Victoria	803
5	declaration 76457	Declaration submitted by employee	07-04-2017 11:38:14	Linda	13
6	declaration 76457	Declaration final approved by supervisor	07-04-2017 11:40:17	Sara	2225
7	declaration 76457	Request payment	11-04-2017 13:03:43	Undefined	2100
8	declaration 76457	Payment handled	13-04-2017 15:30:53	Undefined	2333
9	declaration 76667	Start trip	20-11-2016 23:00:00	Karen	2
10	declaration 76667	End trip	21-12-2016 23:00:00	Karen	3
11	declaration 76667	Permit submitted by employee	28-02-2017 13:42:17	Karen	5
...	...	...	...	...	...
15	declaration 76667	Request payment	09-03-2017 14:14:45	Undefined	2100
16	declaration 76667	Payment handled	13-03-2017 16:31:02	Undefined	2333
...	...	...	...	...	...

Figure 1: Excerpt of an event log, adapted from a real-life event log related to the reimbursement procedure at a Dutch university [25]

Although the common standard for storing event logs is the XES (eXtensible Event Stream) format, they can be converted into relational databases to enable querying with general-purpose languages such as Structured Query Language (SQL). Alternatively, more specialized languages, such as Celonis Process Query Language (PQL), provide process-oriented analysis primitives, i.e., they work directly with event concepts to extract valuable process insights.

3. Related work

Research on the text-to-SQL task has advanced considerably, moving from early deep learning models to pre-trained architectures and, more recently, large language models (LLMs). Initial approaches explored sequence-to-sequence and sequence-to-tree architectures, with hierarchical decoding of SQL queries from natural language [20]. Other works aligned template-based decoding with SQL syntax [2] or introduced grammar-guided decoding [26] to handle the complexity of benchmarks such as Spider [10]. Schema representation became a central challenge, addressed through graph-based en-

¹<https://icpmconference.org/2020/bpi-challenge/>

coding and graph neural networks [22], while intermediate representations such as SemQL mitigated mismatches between natural language intent and SQL syntax [27]. RAT-SQL [6] combined schema encoding and schema linking within a relation-aware attention framework, and GAP [5], pre-trained with multiple synthetic data generation strategies based on BART [28], improved pre-training for column alignment and prediction. Later, RESDSQL [4] leveraged the T5 model [23] with skeleton-aware decoding to enhance SQL generation.

Beyond English, multilingual studies such as mRAT-SQL + GAP [29] translated the Spider dataset into Portuguese and combined it with multilingual pre-trained models (mBART-50), obtaining promising results. With the widespread adoption of LLMs, the field has shifted from fine-tuned task-specific models to prompt engineering. Zero-shot and few-shot strategies have been widely tested with LLMs such as GPT-3.5 Turbo [14], GPT-4, Gemini-1.0 Pro [15], and Llama3-8B Instruct [16]. For example, C3 [30] explored zero-shot prompting with ChatGPT, while DAIL-SQL [7] conducted systematic comparisons across various prompt designs and in-context learning strategies, using the Spider [10] and Bird [9] benchmarks.

Evaluation has also expanded beyond standard metrics. NL2SQL360 [17] proposed a comprehensive framework that considers SQL complexity, query characteristics (e.g., logical operators, subqueries, joins), robustness to natural language variation, and performance in domain-specific databases. Using Spider and Bird, NL2SQL360 compared both PLM-based and LLM-based approaches, offering a broader view of their strengths and limitations.

In the context of process mining, semantic parsing has been investigated to enable natural language interaction with event logs. Some works proposed hybrid pipelines based on traditional NLP techniques and rule-based mappings [31, 32], while recent studies evaluated the use of LLMs such as GPT-4 and Bard in generating SQL queries and hypotheses directly from event logs [33]. In addition, declarative approaches explored how SQL templates can represent process constraints for tasks such as discovery and conformance checking [34].

This paper builds on these advances by combining strategies from prompt-based text-to-SQL research, such as DAIL-SQL [7], with comprehensive evaluation perspectives inspired by NL2SQL360 [17], while focusing on process-oriented data and insights.

4. Method

This section presents the method used to evaluate the text-to-SQL task in process mining domain. It introduces the process mining dataset used, followed by the models and prompt strategies employed to generate SQL statements, and concludes with the evaluation strategies.

Process mining dataset. To evaluate the text-to-SQL task in process mining domain, a bilingual dataset entitled *text₂SQL₄PM* was used². The dataset comprises 1,655 quadruples (NL-PT utterance, NL-EN utterance, SQL statement - PT, SQL statement - EN)³, consisting of 205 base utterances and 1,450 paraphrases, elaborated by humans. Paraphrases were incorporated into the dataset with the intention of performing data augmentation. Initially, the dataset was developed as an exercise for Data Mining and Process Mining classrooms, and it later underwent a curation process conducted by process mining experts. A professional translator was involved to ensure the correctness of translations between languages. An event log related to authorization requests for reimbursement of academic travel expenses [25] was used during the curation process to standardize the focus of the business process dataset.

In addition to the set of quadruples, the dataset owners also provide classifications, referred to as qualifiers. Each quadruple is annotated with ten qualifiers, which are perspective categories⁴ designed to organize the dataset and facilitate the evaluation of the results obtained using this dataset. Beyond their descriptive function, the qualifiers allow a systematic assessment of model robustness, for example, by isolating the effects of SQL complexity or paraphrasing on model accuracy.

Compared to existing text-to-SQL datasets (as WikiSQL [3], Spider [10] and Bird [9]), *text₂SQL₄PM* is distinctive in three ways: it is bilingual; it is domain-specific, being rooted in queries for process mining domain; and it includes detailed perspective-based annotations, which make it particularly relevant for both educational purposes and the benchmarking of models in a domain where natural language interfaces remain underexplored. More details about *text₂SQL₄PM* can be found in [21].

²Available at <https://github.com/pm-usp/text-2-sql>.

³NL - natural language; PT - Brazilian Portuguese. EN - English.

⁴Natural language perspective (wh-question and base-paraphrase categorization); process mining perspective (event-case level, value-domain-generic, perspective-descriptive statistics-conformance categorization); SQL perspective (complexity and SQL operators).

Models and prompt strategies. We evaluated three large language models: GPT-3.5 Turbo [14], Gemini-1.0 Pro [15] and LLama3-8B Instruct [16]. The OpenAI representation (OPR) and the Code representation (CR), as used in DAIL-SQL [7], were chosen as styles for prompt inputs in both languages, Portuguese and English. Examples are provided in Table 1.

The prompts were designed using a zero-shot approach, which only the natural language utterance and the database schema are provided as input, and a few-shot approach, which one example⁵ or some similar examples are included as prompt contexts for inputs. The rationale for combining the zero-shot and few-shot strategies was to contrast the model’s ability to generalize without any prior examples against its performance when guided by semantically related instances.

For a few-shot approach, the examples were provided with the organization information containing the utterance and the SQL statement information. The selection of examples for prompt input was made based on the similarity between the example utterances and the target utterance, using the cosine distance measure. All example utterances and target utterances were encoded to obtain the embeddings using Sentence-BERT [35], and a cosine similarity was calculated between each of the target embeddings and all the example embeddings, selecting the example with the highest result (the most semantically similar). For the selection of examples by similarity, we excluded the paraphrases created for that specific target utterance in order to prevent data leakage and artificially inflated performance. For zero-shot and one-shot evaluations, both OpenAI and Code representations were used, whereas for three-shot and five-shot evaluations, only the Code representation was employed. This choice was made because the initial experiments with one-shot had already demonstrated the superiority of the Code representation prompt.

Evaluations strategies. Two well-known text-to-SQL metrics were adopted to evaluate the models, *Exact Match Accuracy* (EM) and *Execution Accuracy* (EX) [10]. The EM metric is a string-based metric with light structural normalization, assessing whether the SQL statement generated by the model exactly matches the target SQL statement. However, the EM metric ignores values in the condition clauses, which can lead to false positive results; therefore, the EX metric can be used in tandem. The EX metric is an

⁵In this paper, we treat the one-shot strategy as a specific case of the few-shot strategy.

```

– for OpenAI representation (OPR) – zero-shot strategy
#### Complete sqlite SQL query only and with no explanation
#### SQLite SQL tables, with their properties:
#
# event_log(id, activity, timestamp, resource, cost, idcase)
#
#### Calculate the number of events performed by each resource that do not
correspond to the performance of the ‘end trip’ activity.
SELECT

```

```

– for Code representation (CR) – few-shot strategy
/* Some SQL examples are provided based on similar problems: */

/* Answer the following: How many cases went through the ‘end trip’ activity
between 05/02/2017 and 05/12/2017? */
SELECT COUNT(DISTINCT idcase) FROM event_log WHERE activity = ‘End
trip’ AND timestamp BETWEEN ‘2017-05-02’ and ‘2017-05-12’

/* Answer the following: Which cases went through the ‘start trip’ activity and
had not finished by 31/03/2018? */
SELECT idcase FROM event_log where activity = ‘Start trip’ INTERSECT SE-
LECT idcase FROM event_log GROUP BY idcase HAVING MAX(timestamp) >
‘2018-03-31’

/* Answer the following: Which cases went through the ‘start trip’ activity and
had not ended by March 2018? */
SELECT idcase FROM event_log where activity = ‘Start trip’ INTERSECT SE-
LECT idcase FROM event_log GROUP BY idcase HAVING MAX(timestamp) >
‘2018-03-31’

/* Given the following database schema: */
CREATE TABLE “event_log” (
    “id”      int,
    “activity” TEXT COLLATE NOCASE,
    “timestamp” DATETIME,
    “resource” TEXT COLLATE NOCASE,
    “cost”     NUMERIC,
    “idcase”   TEXT COLLATE NOCASE,
    PRIMARY KEY(“id”) )

/* Answer the following: List the cases that arrived at the ‘end trip’ activity
between 2016 and 2017. */

SELECT

```

Table 1: Examples of prompts considering the English language

execution-based metric that compares the results obtained by running the model-generated SQL statement in a database against those of the target SQL statement. Under this metric, SQL statements with different structures are considered correct if they yield the same results. However, a limitation, is that semantically different statements may also be judged correct when they coincidentally produce the same output (e.g., when both queries return an empty result set).

To guide a more detailed analysis of the results, we proposed SQL pattern templates expressed in a grammar-like representation. All SQL pattern templates are presented on the github repository⁵ and the details of each symbol of this grammar-like structure are explained in Table 2. The introduction of these templates accounts for the complexities of SQL statements and provides a new perspective for analysis. Their design was inspired by the NL2SQL360 study [17], which categorized SQL statements in its dataset based on specific SQL characteristics. Unlike NL2SQL360, however, our approach distinguishes templates by SQL complexity, rather than using a general categorization. While NL2SQL360 focused on the presence or absence of elements such as logical connectors, subqueries, JOIN operators, and ORDER BY operators, our method emphasizes complexity-aware differentiation.

5. Results and analysis

This section presents results and analysis of experiments on large language models, using the *text₂SQL₄PM* dataset as a benchmark. The results are presented in terms of: performance of models and prompt strategies under the EM and EX metrics; performance with respect to SQL complexity through SQL template-based analyses; and the connection between increasing SQL query complexity and the challenges of information retrieval in process mining domain.

EM and EX metrics results. Table 3 presents the results of the EM and EX metric for zero-shot, one-shot using OpenAI representation and for zero-shot, one-shot, three-shot, and five-shot using Code representation. For zero-shot, the OpenAI representation achieves higher EM scores for GPT-3.5 Turbo

⁵Available at <https://anonymous.4open.science/r/text-to-sql-process-mining-114C>

Symbol	Description
+	one or more
*	zero or one
Sc	a concept of interest on SELECT clause
Ac	an aggregation on a concept of interest on SELECT clause
Acc	an aggregation on a concept of interest grouped by another concept on SELECT clause
Fc	a filter on a concept presented on WHERE clause or if it appears after Gc symbol it represents a filter on HAVING clause
FSc	a filter on a concept with SUBSELECT presented on WHERE clause or HAVING clause if it appears after Gc symbol
Gc	a concept grouped by GROUP clause
SIE	a combination of set operations with INTERSECT or EXCEPT clauses between two queries
SU	a combination of a union operation with UNION clause between two queries
RS	a SUBSELECT result on a JOIN or FROM clause
SUE	a combination of set operations with EXCEPT clause or a union combination with UNION clause between two queries
WS	a SQL statement that contains WITH clause

Table 2: Details and descriptions of grammar symbols of SQL pattern templates - concept: an attribute that have meaning on process mining domain (resource, activity, event, cost, timestamp, case). Reproduced from [13]

in both languages, Portuguese and English. Regarding the EX metric, the OpenAI representation performs better for GPT-3.5 Turbo in English, while Code representation shows better results for GPT-3.5 Turbo in Portuguese. For few-shot strategies (one-shot, three-shot, and five-shot), the Code representation prevails with better results for GPT-3.5 Turbo for the EM and EX metric in both languages. Furthermore, performance improves progressively as the number of shots increases, as providing more examples in the prompt context, enables the models to generate SQL statements similar to the provided examples, thereby enhancing accuracy on both metrics.

The highest EM scores were achieved by GPT-3.5 Turbo with the Code representation under the five-shot strategy, reaching 48.46% in English and 45.14% in Portuguese. Similarly, the EX metric scores were achieved by GPT-3.5 Turbo with Code representation under the five-shot strategy, reaching 57.16% in English and 56.25% in Portuguese. Although the growth

			GPT-3.5 Turbo		Gemini-1.0 Pro		Llama3-8B	
			OPR	CR	OPR	CR	OPR	CR
EM	EN	zero-shot	25.50	14.68	22.66	5.20	16.37	13.78
		one-shot	37.16	40.12	24.89	28.04	30.45	24.95
		three-shot	-	46.10	-	31.36	-	30.76
		five-shot	-	48.46	-	32.87	-	34.80
	PT	zero-shot	24.77	16.92	21.15	5.38	15.65	11.90
		one-shot	36.01	38.01	27.43	27.67	23.99	19.03
		three-shot	-	42.66	-	32.57	-	21.51
		five-shot	-	45.14	-	33.90	-	24.59
EX	EN	zero-shot	47.61	46.47	39.70	37.52	36.13	39.09
		one-shot	47.85	54.56	38.67	47.73	39.40	47.79
		three-shot	-	55.53	-	51.54	-	51.12
		five-shot	-	57.16	-	53.78	-	53.53
	PT	zero-shot	44.53	45.74	37.16	40.18	35.77	36.01
		one-shot	47.79	52.63	42.66	49.49	35.29	43.50
		three-shot	-	55.23	-	54.26	-	49.00
		five-shot	-	56.25	-	55.89	-	50.21

Table 3: Results of EM and EX metrics for zero-shot and few-shot prompt strategies using OpenAI (OPR) and Code (CR) representations, in Portuguese (PT) and English (EN)

pattern is similar in EN and PT, absolute scores are consistently higher in English, reflecting both the broader lexical-semantic coverage of LLMs in English and the fact that SQL statements itself are expressed in English keywords, which also appear in the input utterances. Notably, however, in the EX metric, this gap nearly vanishes in the best settings, suggesting that execution-level generalization mitigates the language disparity.

SQL pattern templates results and analysis. An analysis using SQL pattern templates was performed to identify the main challenges associated with variations in SQL statements within dataset *text₂SQL₄PM*. The challenges encountered are related to the following (examples are provided in Table 4):

- domain-specific terms/constructs (DS);
- interpretation of values for filtering results (IVFR);
- additional or missing attributes in the SELECT clause (AMS);

	Utterance	Challenge
DS	Which declarations were rejected by a pre-approver other than Douglas ?	Interpreting the term declarations as cases, mapping a rejection by a pre-approver to the value ‘Declaration rejected by pre-approver’, and identifying Douglas as a resource.
IVFR	How many times were ‘ permit ’ activities performed in each case?	Distinguishing among multiple types of permit-related activities (e.g., permit submitted by employee).
AMS	Which resource was most active in the ‘declaration approved by administration’ activity?	Avoiding misinterpretation that leads to counting resources in addition to retrieving the most active one.
PMC	What is the average duration of each activity?	Requiring knowledge of each case, including the sequence and duration of activities, before computing the overall average.
IEA	How many activities were associated with each resource?	Clarifying the meaning of activities to prevent ambiguity in interpretation.

Table 4: Examples of each challenge - DS: domain-specific terms; IVFR: interpretation of values for filtering results; AMS: additional or missing attributes in SELECT clause; PMC: process mining case concept; IEA: interchangeable use of the terms ‘event’ and ‘activity’. Revised from [13]

- understanding and interpretation of the process mining concept – *case* (PMC);
- interchangeable use of the terms *activity* and *event* in process mining, which complicates the selection of the appropriate term in various contexts (IEA).

The performance analysis of language models and prompt strategies under the SQL pattern templates was carried out considering the categorization of one of the qualifiers in the *text₂SQL₄PM* dataset — the qualifier 8 (complexity) from the SQL perspective. This qualifier classifies dataset instances according to the complexity of the SQL associated with the input utterance,

using the categories easy, medium, hard, extra hard, and no_hardness⁶ [10]. In this set of analyses, only illustrations for the setup “GPT-3.5 Turbo with Code Representation” are presented for the EM and EX metrics, in both Portuguese and English, as this achieved the best overall results.

Easy class: For this complexity level, five SQL pattern templates were identified: Sc, ScFc, ScGcFc⁺, Ac, and AcFc. The results observed in Figure 2 show that, for the EM metric in Portuguese and English, the few-shot strategy significantly outperformed the zero-shot strategy. The most notable improvement can be observed for the Ac template, which increased from 2.63% (zero-shot) to 92.11% (three-shot) in English, and from 10.53% (zero-shot) to 94.74% (five-shot) in Portuguese. For the other templates, the language models also showed improvements when moving from zero-shot to few-shot, although some exhibited slight decreases as the number of shots increased. For the EX metric in both languages, the few-shot strategy generally led to increases for the Sc, ScGcFc⁺, Ac, and AcFc templates. However, for template ScFc, the Portuguese results remained nearly unchanged as the number of shots increased, whereas in English they decreased from zero-shot to few-shot. Some syntactic and semantic observations can be highlighted to better understand the challenges found in the SQL-easy class:

- for the Ac template (SELECT clause), the large variation in the EM metric when moving from zero-shot to few-shot is due to the addition of an alias (AS) in the projection (SELECT clause), which explains the higher results observed in the EX metric;
- the higher results of the EX metric compared to the EM metric for the ScFc template (SELECT and WHERE clauses) refer to challenges in interpreting natural language utterance terms for projection and filtering, particularly for date values (challenge IVFR, cf. Table 4);
- for the ScGcFc⁺ template (SELECT, GROUP BY and HAVING clauses), challenges were encountered in understanding the process mining *case* concept (challenge PMC, cf. Table 4);

⁶no_hardness: a class which a gold SQL statement that could not be classified due limitations of the scripts of Spider benchmark (ex.: use of NULL and IS NULL operators, use of function STRFTIME, etc).

- for the AcFc template (SELECT and WHERE clauses), challenges were encountered with additional attributes unnecessarily included in the SELECT clause (challenge AMS, cf. Table 4);
- for all templates, the language models struggle with domain-specific terms which there are no explicit connections between the items of the database schema and the utterance terms (challenge DS, cf. Table 4).

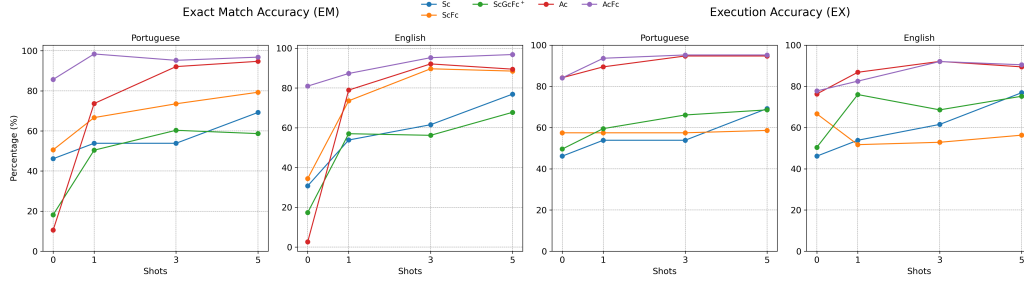


Figure 2: Results of the EM and EX metrics for GPT-3.5 Turbo with Code representation using zero-shot, one-shot, three-shot, and five-shot on SQL pattern templates of the easy class

Medium class: For this complexity level, seven SQL pattern templates were identified: Sc^+ , $ScGc$, Sc^+Fc^+ , Sc^+GcFc^+ , Acc^+Gc , $Ac(c^+)*Fc^+(Gc)^*$ and $(Ac)^+c(Fc)*GcFc$. The results observed in Figure 3 show that, for the EM metric in Portuguese and English, the few-shot strategy significantly outperformed zero-shot. In particular, the $ScGc$ and Acc^+Gc templates, for which the language models yielded no correct mappings or only negligible results under the zero-shot strategy, surpassed 60% when the few-shot strategy was applied. For the EX metric, there was a significant improvement for the $ScGc$ template, particularly in Portuguese, from zero-shot to one-shot, with results remaining unchanged from three-shot and five-shot. Highlights include the results above 90% with the few-shot strategy for Acc^+Gc and $Ac(c^+)*Fc^+(Gc)^*$ templates in Portuguese and for Acc^+Gc , $Ac(c^+)*Fc^+(Gc)^*$ and $(Ac)^+c(Fc)*GcFc$ templates in English. Some frequency-related, syntactic and semantic observations can be highlighted to better understand the challenges found in the SQL-medium class:

- the large variation in the EM metric for the $ScGc$ template (SELECT and GROUP BY clauses) from zero-shot to few-shot is partly due to the small number of samples of this SQL pattern template;

- for the Acc^+Gc , $\text{Ac}(\text{c}^+)\text{Fc}^+(\text{Gc})^*$ and $(\text{Ac})^+\text{c}(\text{Fc})^*\text{GcFc}$ templates (different uses and combinations of the SELECT, WHERE, GROUP BY and HAVING clauses) the large variation in the EM metric when moving from zero-shot to few-shot is due to the addition of an alias (AS) in the projection, which explains the higher results of the EX metric.
- the lower results for the Sc^+GcFc^+ template (SELECT, GROUP BY and HAVING clauses) highlight challenges related to the implicit knowledge of the *case* concept (challenge PMC, cf. Table 4), as well as issues caused by the inclusion of unnecessary additional attributes in the SELECT clause (challenge AMS, cf. Table 4);
- for all templates, the language models struggle with challenges related to domain-specific terms (challenge DS cf. Table 4).

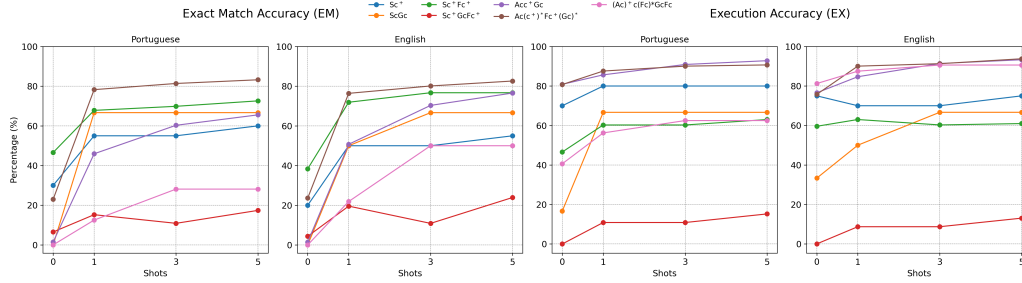


Figure 3: Results of the EM and EX metrics for GPT-3.5 Turbo with Code representation using zero-shot, one-shot, three-shot, and five-shot on SQL pattern templates of the medium class

Hard class: For this complexity level, seven SQL pattern templates were found: $\text{Sc}^+\text{Fc}^+(\text{Gc})^*$, ScFSc , ScFcGcFc , ScGcFSc , AcFSc , AccFc^+Gc , and SIE . The results observed in Figure 4 show that, for the EM metric, significant improvements were observed when moving from zero-shot to few-shot, for $\text{Sc}^+\text{Fc}^+(\text{Gc})^*$ and AccFc^+Gc templates in both Portuguese and English. Additionally, a significant improvement for the ScFSc template in Portuguese was also observed. Lower results were observed for ScGcFSc and SIE templates in zero-shot and few-shot strategies in both languages. For the EX metric and for the ScFcGcFc template, performance decreased as the number of shots increased in Portuguese, while in English, it achieved results of 100% with one-shot but declined as the number of shots increased to three

and five. Some frequency-related, syntactic and semantic observations can be highlighted to better understand the challenges found in the SQL-hard class:

- the inconsistent variation for the ScFcGcFc and AcFSc templates (different uses and combinations of SELECT/SUBSELECT, WHERE, and GROUP BY clauses) from zero-shot to few-shot is partly due to the small number of samples;
- the ScFSc, ScGcFSc and AcFSc templates (different uses and combinations of the SELECT/SUBSELECT, GROUP BY and HAVING clauses), which involve more complex SQL statements with SUBSELECT, present a particular challenge for utterance interpretation when tied boundary results occur (challenge IVFR, cf. Table 4). For example, in the utterance “Which cases have the greatest number of events?”, the interpretation must account for inclusive tie handling, ensuring that all cases sharing the maximum value are returned;
- for the SIE template (set operations clause – INTERSECT or EXCEPT), the language models struggle with the use of the INTERSECT and EXCEPT SQL operators, often seeming to avoid them in SQL generation.
- for all templates the language models struggle with domain-specific terms challenges(challenge DS cf. Table 4).

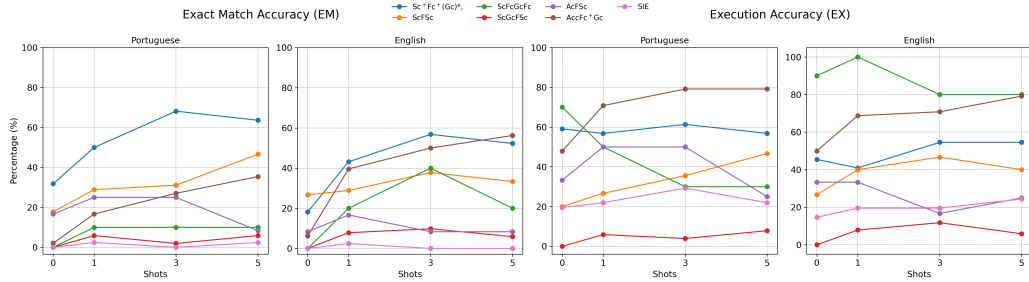


Figure 4: Results of the EM and EX metrics for GPT-3.5 Turbo with Code representation using zero-shot, one-shot, three-shot, and five-shot on SQL pattern templates of the hard class

Extra hard class: For this complexity level, seven SQL pattern templates were identified: $\text{ScFc}^+(\text{Gc})^*$, $\text{ScFSc}(\text{Fc})^*(\text{Gc})^*$, $\text{Sc}(\text{Fc})^*\text{GcFSc}$, $\text{Acc}^+\text{Fc}^+\text{Gc}^+$, AcFSc , $\text{AccGc}(\text{FSc})^*$, and SU . The results observed in Figure 5 show that, regarding the EM metric, the $\text{ScFSc}(\text{Fc})^*(\text{Gc})^*$, $\text{Acc}^+\text{Fc}^+\text{Gc}^+$ and $\text{AccGc}(\text{FSc})^*$ templates, the language models produced no results under the zero-shot strategy (0%), started to yield non-zero results with few-shot in both Portuguese and English. For the $\text{Sc}(\text{Fc})^*\text{GcFSc}$ template, few-shot also enabled results in Portuguese, while in English the language models consistently remained at 0% regardless of the prompt strategy. For the AcFSc and SU templates, language models obtained 0% in both languages for all prompt strategies. For the EX metric, the language models stands out for $\text{Acc}^+\text{Fc}^+\text{Gc}^+$ template, reaching 100% with three-shot and five-shot in Portuguese and with one-shot and three-shot in English. Additionally, few-shot enabled the language models to yield non-zero results for the $\text{ScFSc}(\text{Fc})^*(\text{Gc})^*$ and AcFSc templates. However, for the $\text{Sc}(\text{Fc})^*\text{GcFSc}$ template, the results remained at 0% in English under all prompt strategies. Some frequency-related, syntactic and semantic observations can be highlighted to better understand the challenges found in the SQL-extra hard class:

- for the $\text{Acc}^+\text{Fc}^+\text{Gc}^+$ and $\text{AccGc}(\text{FSc})^*$ templates (use and combinations of the SELECT/SUBSELECT, WHERE, GROUP BY and HAVING clauses), the lower EM metric results compared to the EX metric are mainly explained by the addition of an alias ‘AS’ in the projection clause;
- For the AcFSc template (SELECT/SUBSELECT clauses), no correct mappings were obtained in the EM metric across all shot strategies. This is explained by the presence of the ‘DISTINCT’ keyword within the ‘NOT IN’ clause in the reference SQL statement (e.g., NOT IN (SELECT DISTINCT case)), which is unnecessary and therefore omitted in the model-generated SQL statements. This mismatch is reflected in the EX metric, where some variation is observed as the number of shots increases, mainly due to the small number of samples in this template.
- For the $\text{ScFc}^+(\text{Gc})^*$ template (SELECT, WHERE and GROUP BY clauses), challenges arise in the interpretation of the *activity* term (challenge IEA, cf. Table 4) and in the inclusion of additional values in the SELECT clause (challenge AMS, cf. Table 4).

- For the $ScFSc(Fc)^*(Gc)^*$ template (SELECT/SUBSELECT, WHERE and GROUP BY clauses), challenges arise from the implicit knowledge of the *case* concept (challenge PMC, cf. Table 4), while for the $Sc(Fc)^*GcFSc$ template (SELECT/SUBSELECT, WHERE, GROUP BY and HAVING clause), challenges occur in utterance interpretation involving tied boundary results.
- Regarding the SU template (set operation clause – UNION), which involves the UNION operation, the language models seems to avoid incorporating this construct in the generated SQL statements. This behavior is evident in the EM metric, when compared to the EX metric, which may return unexpected results.
- All templates struggle with challenges related to domain-specific terms (challenge DS, cf. Table 4).

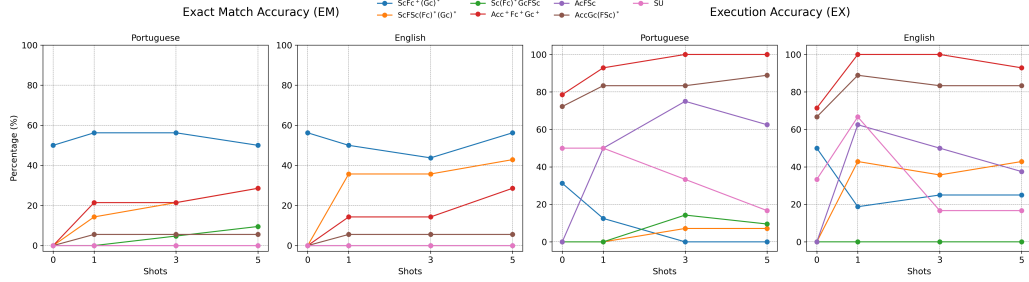


Figure 5: Results of the EM and EX metrics for GPT-3.5 Turbo with Code representation using zero-shot, one-shot, three-shot, and five-shot on SQL pattern templates of the extra hard class

‘No_hardness’ class: For this complexity level, ten SQL pattern templates were found: $Sc^+Fc^+(Gc)^*$, $Sc^+Gc(Fc)^*$, $Sc^+(Fc)^*FSc(Gc)^*$, $Sc^+RS(Fc)^*$, $Ac(c)^*Fc^+(Gc)^*$, $AccFScGc$, $AccGc$, $Ac(c)^*RS(Fc)^*(Gc)^*$, WS and $S(UE)^+$. The results illustrated in Figure 6 present only the EX metric, due to limitations of the Spider scripts used for the evaluation, which prevented the EM metric from being computed for this category. For the WS template, the language models did not produce correct mappings in either English or Portuguese, regardless of the prompt strategy. Although for $Sc^+(Fc)^*FSc(Gc)^*$, Sc^+RS and $AccFScGc$ templates, they produced some correct mappings, the

performance was negligible. For $Sc^+Gc(Fc)^*$ and $S(UE)^+$ templates, the few-shot strategy enabled non-zero results, in contrast to the consistent absence of correct mappings under the zero-shot setting, in both Portuguese and English.

Besides the previously mentioned issues in other complexity classes and also present in the ‘no hardness’ class, one challenge becomes evident in the ‘no hardness’ class across all SQL pattern templates. The majority of utterances in that category require a fundamental understanding of the process mining *case* concept (challenge PMC cf. Table 4). This knowledge implies that models must understand that a case has a start, an end, and events in between that occur sequentially, ordered by their occurrence time. This notion, implicit in the utterances, consequently implies that complex SQL statements must be generated in response, which occurs practically in all SQL pattern templates of this category except the AccGc template.

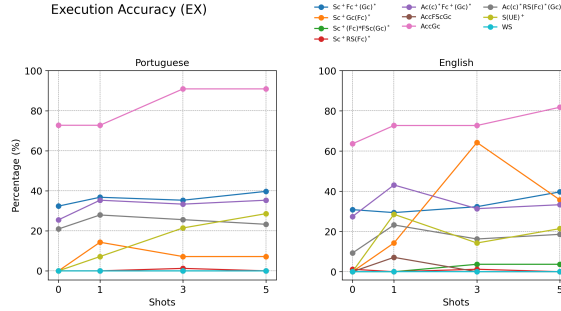


Figure 6: Results of EX metrics for GPT-3.5 Turbo with Code representation using zero-shot, one-shot, three-shot, and five-shot on SQL pattern templates of the ‘no hardness’ class

Although basic prompting strategies allow for a more direct evaluation of each LLMs capabilities, the results in Table 3 indicate that these strategies alone are insufficient to create a viable text-to-SQL implementation for the process mining domain. Furthermore, as shown in Figure 6, SQL statements categorized as ‘no hardness’, those with greater potential usability in this domain, yielded the poorest results. While larger models might mitigate some of these shortcomings, relying solely on scale is neither sustainable nor sufficient. Therefore, developing a systemic solution requires advancing towards more detailed and informative prompts, possibly combined with complementary strategies that go beyond model size.

SQL query complexity vs. information retrieval in process mining domain. The classification into easy, medium, hard, extra hard, and no_hardness reveals a progression of challenges in SQL formulation and, in fact, also in utterance interpretation, and conceptual understanding of process mining. At each level, queries become structurally denser, with more clauses, operators, and combinations, and demand greater articulation between linguistic semantics and domain concepts such as cases, events, resources, activities, costs, and time. This progression also reflects in the SQL pattern templates, which evolve from simple structures to complex combinations. This progression can be illustrated across the different classes:

- Easy class: Even the simplest queries are not trivial. They include direct utterances about the entire log or basic statistics, but also situations where it is already necessary to relate events, cases, and resources with simple filters. Typical examples include calculating “the number of events in a given period” or “counting the resources involved in each case”. The central challenge lies in correctly interpreting basic domain concepts, such as distinguishing the label of an activity from its occurrence (event) within a process instance (case).
- Medium class: At this level, queries articulate multiple perspectives (events, cases, resources, and costs) through operations such as ordering, grouping, or conditional restrictions. An example is measuring “the frequency of an activity performed by a resource within a given time interval”. The step beyond the easy level lies in the need to handle the use of the same concept across multiple clauses of the SQL statement, which requires greater semantic precision in aligning the input utterance with the corresponding SQL statement.
- Hard class: Queries become more intricate, requiring detailed reasoning about exceptions, edge cases, and implicit process knowledge. Examples include “identifying cases that did not perform a given activity”, “selecting the highest-cost events”, or “counting repetitions of activities above a threshold”. The difficulty lies not only in the linguistic ambiguity of terms such as “did not perform” or “the highest”, but also in domain interpretation, for example, what it means for a case to “not contain” a given activity within a business process.
- Extra hard class: Complexity continues to increase gradually. At this level, queries combine multiple perspectives, composite conditions, and

specific orderings. Common examples include “resources that did not perform a specific activity” or “cases that went through a specific activity that must be displayed in order according to the number of resources involved”. Such formulations illustrate a stage in which linguistic and conceptual ambiguities become more prevalent.

- **No_hardness class:** This residual category groups queries that do not fit into the previous levels but still reveal important challenges. Although natural language utterances may often appear simple, such as “calculate the average time between activities” or “identify the first event of each case”, the structure of the schema and the need to navigate across events within the same case, consider multiple cases, and relate activities to resources make the SQL statements formulation significantly complex, involving subqueries, groupings, and orderings. Thus, the no_hardness class shows that difficulty does not stem only from the apparent sophistication of the utterance but also from how log elements must be combined to produce the correct answer.

The analysis of the five classes reveals a central trade-off: the greater the structural and conceptual complexity of queries, the higher their potential to extract valuable information for process mining, but also the greater the risk of interpretive errors. Thus, the increase in complexity is not merely technical (more SQL clauses), but primarily semantic and conceptual, requiring fine-grained alignment between natural language utterances, SQL logic, and the fundamental concepts of process mining.

6. Conclusions

In this paper, we investigate the usability and feasibility of the text-to-SQL task with LLMs applied to the process mining domain. Using a bilingual dataset of utterance–SQL pairs in Portuguese and English, called *text₂SQL₄PM*, we evaluated three large language-based models (GPT-3.5 Turbo, Gemini-1.0 Pro, and Llama3-8B Instruct) under OpenAI and Code representations with zero-shot and few-shot prompt strategies. SQL statements in the *text₂SQL₄PM* dataset were categorized by SQL pattern templates and complexity, allowing a structured analysis in combination with two widely adopted evaluation metrics: Exact Match Accuracy (EM) and Execution Accuracy (EX). The experimental results showed that GPT-3.5

Turbo with Code representation produced the highest performance, reaching 48.46% and 45.14% EM in English and Portuguese, respectively, and 57.16% and 56.25% EX in the same languages, all under the five-shot strategy.

Beyond these performance figures, our cross-analysis of SQL complexity classes, SQL pattern templates, and information retrieval aspects in process mining revealed important limitations of language models when restricted to basic prompt engineering strategies. These findings highlight that applying text-to-SQL mapping to the process mining domain, still highly specific and underexplored, requires not only better prompting approaches but also deeper alignment with domain concepts and structures.

Although the models used represent earlier and simpler versions of their respective families (selected mainly for affordability), this should not be interpreted as outdated results. The purpose of this study is not to provide definitive performance figures, especially given the rapid evolution of LLMs, but rather to illustrate how text-to-SQL evaluation can be systematically conducted in process mining domain. To this end, both the dataset and the experimental design have been easily reproducible, serving as a baseline benchmark for testing newer models whenever desired. Although larger models with more parameters may yield different results, likely correlated with those observed here, we hypothesize that the primary challenges identified in our analysis are less a matter of scale and more related to the need for advanced prompt engineering strategies tailored to process mining concepts. Ideally, such strategies should be integrated into reasoning methods capable of generalizing across different domains of business processes, a direction that will guide the next steps in this line of research.

A potential limitation of our work, at least in terms of practical applicability, concerns the focus on SQL rather than on PQL, the query language specifically designed for process mining. Extending the same methodology to PQL would indeed be valuable, but would require substantial additional effort, particularly to construct a dataset equivalent to *text₂SQL₄PM* in PQL. Such an endeavor would be considerably more complex, as PQL is less widespread than SQL and less familiar to most practitioners, meaning that only a small group of specialists could realistically contribute to dataset creation. Such constraints would likely result in a labor-intensive process with reduced diversity of utterances. For these reasons, we regard the use of SQL not as a limitation but as a deliberate scope choice, enabling reproducibility and broader participation at this initial stage of research. A natural continuation of this paper would be to extend the approach to text-to-PQL.

References

- [1] T. Yu, Z. Li, Z. Zhang, R. Zhang, D. Radev, TypeSQL: Knowledge-Based Type-Aware Neural Text-to-SQL Generation, in: Proc. of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers), Association for Computational Linguistics, New Orleans, Louisiana, 2018, pp. 588–594.
- [2] X. Xu, C. Liu, D. Song, SQLNet: Generating Structured Queries From Natural Language Without Reinforcement Learning, CoRR abs/1711.04436 (2017).
- [3] V. Zhong, C. Xiong, R. Socher, Seq2SQL: Generating Structured Queries from Natural Language using Reinforcement Learning, CoRR abs/1709.00103 (2017).
- [4] H. Li, J. Zhang, C. Li, H. Chen, RESDSQL: Decoupling Schema Linking and Skeleton Parsing for Text-to-SQL, 2023.
- [5] P. Shi, P. Ng, Z. Wang, H. Zhu, A. H. Li, J. Wang, C. N. Santos, B. Xiang, Learning contextual representations for semantic parsing with generation-augmented pre-training, Proc. of the AAAI Conference on Artificial Intelligence 35 (2021) 13806–13814.
- [6] B. Wang, R. Shin, X. Liu, O. Polozov, M. Richardson, RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers, in: Proc. of the 58th Annual Meeting of the Association for Computational Linguistics, Association for Computational Linguistics, Online, 2020, pp. 7567–7578.
- [7] D. Gao, H. Wang, Y. Li, X. Sun, Y. Qian, B. Ding, J. Zhou, Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation, CoRR abs/2308.15363 (2023).
- [8] M. Pourreza, D. Rafiei, DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction, 2023. arXiv:2304.11015.
- [9] J. Li, B. Hui, G. Qu, B. Li, J. Yang, B. Li, B. Wang, B. Qin, R. Cao, R. Geng, N. Huo, C. Ma, K. C. C. Chang, F. Huang, R. Cheng, Y. Li, Can LLM Already Serve as A Database Interface?

- A BIG Bench for Large-Scale Database Grounded Text-to-SQLs, 2023. [arXiv:2305.03111](#).
- [10] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, D. Radev, Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task, in: Proc. of the 2018 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, Brussels, Belgium, 2018, pp. 3911–3921.
 - [11] W. M. P. van der Aalst, Process Mining: Data Science in Action, 2 ed., Springer, Heidelberg, 2016.
 - [12] A. Berti, D. Schuster, W. M. P. van der Aalst, Abstractions, Scenarios, and Prompt Definitions for Process Mining with LLMs: A Case Study, in: J. D. Weerd, L. Pufahl (Eds.), Business Process Management Workshops, Springer Nature Switzerland, Cham, 2024, pp. 427–439.
 - [13] B. Y. Yamate, T. R. Neubauer, M. Fantinato, S. M. Peres, Applying Text-to-SQL in process mining: Leveraging natural language for data insights, in: Emerging Trends in Information Systems and Technologies, Springer, 2025. To appear.
 - [14] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. F. Christiano, J. Leike, R. Lowe, Training language models to follow instructions with human feedback, in: S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, A. Oh (Eds.), Advances in Neural Inf. Processing Systems, volume 35, Curran Associates, Inc., 2022, pp. 27730–27744.
 - [15] G. Team, Gemini: A Family of Highly Capable Multimodal Models, [arXiv 2312.11805](#) (2024).
 - [16] L. Team, The Llama 3 Herd of Models, [arXiv abs/2407.21783](#) (2024).
 - [17] B. Li, Y. Luo, C. Chai, G. Li, N. Tang, The Dawn of Natural Language to SQL: Are We Fully Ready?, Proc. of the VLDB Endowment 17 (2024) 3318–3331.
 - [18] J. Eisenstein, Natural Language Processing, 2018.

- [19] I. Goodfellow, Y. Bengio, A. Courville, Deep Learning, MIT Press, 2016.
- [20] L. Dong, M. Lapata, Language to Logical Form with Neural Attention, CoRR abs/1601.01280 (2016).
- [21] B. Y. Yamate, T. R. Neubauer, M. Fantinato, S. M. Peres, Text-to-SQL oriented to the process mining domain: A PT-EN dataset for query translation, 2025. Available at <https://github.com/pm-usp/text-2-sql>.
- [22] B. Bogin, M. Gardner, J. Berant, Representing Schema Structure with Graph Neural Networks for Text-to-SQL Parsing, in: Proc. of the 57th Annual Meeting of the Association for Computational Linguistics, Association for Computational Linguistics, Florence, Italy, 2019, pp. 4560–4565.
- [23] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, P. J. Liu, Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer, J. Mach. Learn. Res. 21 (2020).
- [24] W. M. P. van der Aalst, J. Carmona (Eds.), Process Mining Handbook, volume 448 of *Lecture Notes in Business Inf. Processing*, Springer, 2022.
- [25] B. van Dongen, BPI Challenge 2020: Request For Payment, 2020. URL: https://data.4tu.nl/articles/dataset/BPI_Challenge_2020_Request_For_Payment/12706886/1.
- [26] T. Yu, M. Yasunaga, K. Yang, R. Zhang, D. Wang, Z. Li, D. R. Radev, SyntaxSQLNet: Syntax Tree Networks for Complex and Cross-Domain Text-to-SQL Task, in: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, Brussels, Belgium, 2018, pp. 1653–1663. URL: <https://aclanthology.org/D18-1193>. doi:10.18653/v1/D18-1193.
- [27] J. Guo, Z. Zhan, Y. Gao, Y. Xiao, J. Lou, T. Liu, D. Zhang, Towards Complex Text-to-SQL in Cross-Domain Database with Intermediate Representation, in: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, Association for Computational Linguistics, Florence, Italy, 2019, pp. 4524–4535. URL: <https://aclanthology.org/P19-1444>. doi:10.18653/v1/P19-1444.

- [28] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, L. Zettlemoyer, BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension, in: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, Association for Computational Linguistics, Online, 2020, pp. 7871–7880. URL: <https://aclanthology.org/2020.acl-main.703>. doi:10.18653/v1/2020.acl-main.703.
- [29] M. A. José, F. G. Cozman, mRAT-SQL+GAP: A portuguese text-to-sql transformer, in: Anais da X Brazilian Conference on Intelligent Systems, SBC, Porto Alegre, RS, Brasil, 2021.
- [30] X. Dong, C. Zhang, Y. Ge, Y. Mao, Y. Gao, lu Chen, J. Lin, D. Lou, C3: Zero-shot Text-to-SQL with ChatGPT, 2023. [arXiv:2307.07306](https://arxiv.org/abs/2307.07306).
- [31] L. Barbieri, E. R. M. Madeira, K. Stroeh, W. M. P. van der Aalst, Towards a Natural Language Conversational Interface for Process Mining, in: J. Munoz-Gama, X. Lu (Eds.), Process Mining Workshops, Springer International Publishing, Cham, 2022, pp. 268–280.
- [32] M. Kobeissi, N. Assy, W. Gaaloul, B. Defude, B. Haidar, An Intent-Based Natural Language Interface for Querying Process Execution Data, in: 2021 3rd International Conference on Process Mining (ICPM), 2021, pp. 152–159. doi:10.1109/ICPM53251.2021.9576850.
- [33] A. Berti, M. S. Qafari, Leveraging Large Language Models (LLMs) for Process Mining (Technical Report), 2023. URL: <https://arxiv.org/abs/2307.12701>. [arXiv:2307.12701](https://arxiv.org/abs/2307.12701).
- [34] S. Schönig, A. Rogge-Solti, C. Cabanillas, S. Jablonski, J. Mendling”, Efficient and Customisable Declarative Process Mining with SQL, in: S. Nurcan, P. Soffer, M. Bajec, J. Eder (Eds.), Advanced Information Systems Engineering, Springer International Publishing, Cham, 2016, pp. 290–305.
- [35] N. Reimers, I. Gurevych, Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks, in: Proc. of the 2019 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, 2019, pp. 123–130.